

LABORATORY ACTIVITY NO. 1 REPORT

Real-Time Multisensor Room Monitoring System on ESP32 with FreeRTOS

Course: BCA152 Microcontrollers	Date: September 2026
Student: Michael Cadiz	Instructor: Paul Rodolf P. Castor
Repository: github.com/mccd1/bca152-freertos-multisensor	Environment: PlatformIO (ESP-IDF + FreeRTOS)

1. Problem and Requirements

In typical beginner microcontroller projects using Arduino, all code runs inside a single `loop()` function. The main problem with this approach is that when a sensor like the DHT22 takes 2 seconds to read, the whole system freezes. During this time, the microcontroller cannot read button presses, rotary encoder turns, or motion signals immediately.

To solve this, this laboratory activity uses the native ESP-IDF framework with FreeRTOS. By dividing the system into independent tasks with assigned priorities, the ESP32 can continuously read user inputs and trigger alarms without being blocked by slower sensor readings.

Components and Simulated Wokwi Setup:

- **ESP32 DevKit:** Main dual-core microcontroller running FreeRTOS tasks.
- **DHT22 (GPIO 4):** Reads ambient room temperature (°C) and relative humidity (%).
- **Photoresistor LDR (GPIO 34 / ADC1_CH6):** Reads ambient light level scaled to 0–100%.
- **PIR Motion Sensor (GPIO 13):** Detects motion to keep the system active or wake it up.
- **KY-040 Rotary Encoder (GPIO 18, 19, 5):** Allows the user to navigate across display pages.
- **SSD1306 OLED (GPIO 21, 22 - I2C):** Shows real-time sensor data and system status.
- **Active Buzzer (GPIO 15):** Sounds an alert when temperature goes below 18°C or above 30°C.

2. System Architecture and Design

I designed the software using modular layers so that hardware-specific code (drivers) is separated from decision logic (like state transitions and alarm checks). This made it possible to write unit tests for the logic without depending on hardware.

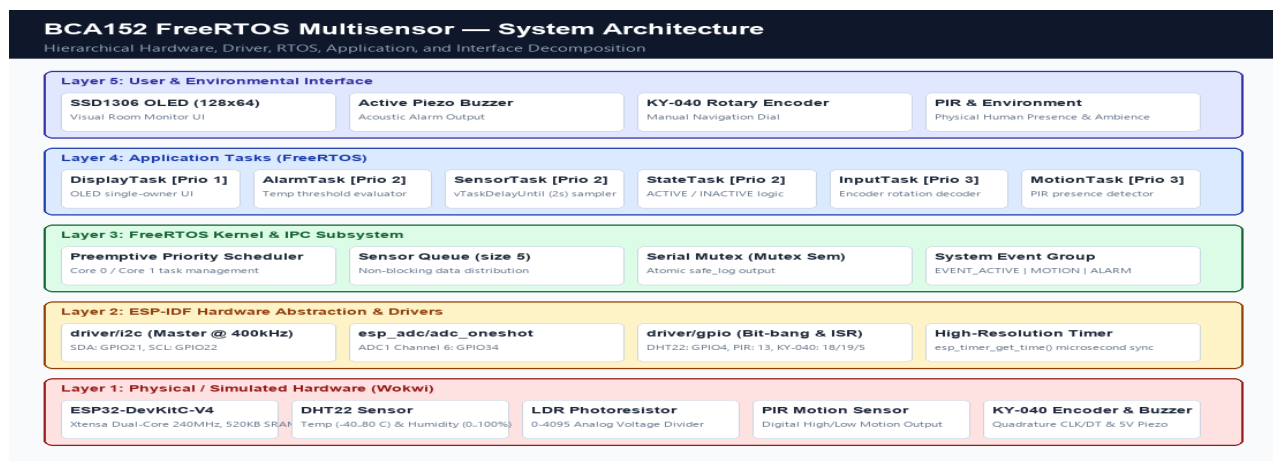


Figure 1: System architecture showing sensor inputs, FreeRTOS tasks, and outputs.

Power Management State Machine: The system has two operating states: ACTIVE and INACTIVE. When active, the OLED display stays on and updates normally. If no motion is detected for 15 seconds, the state machine switches the system to INACTIVE and turns off the OLED to save power. When motion is detected again via the PIR sensor, it immediately returns to ACTIVE.

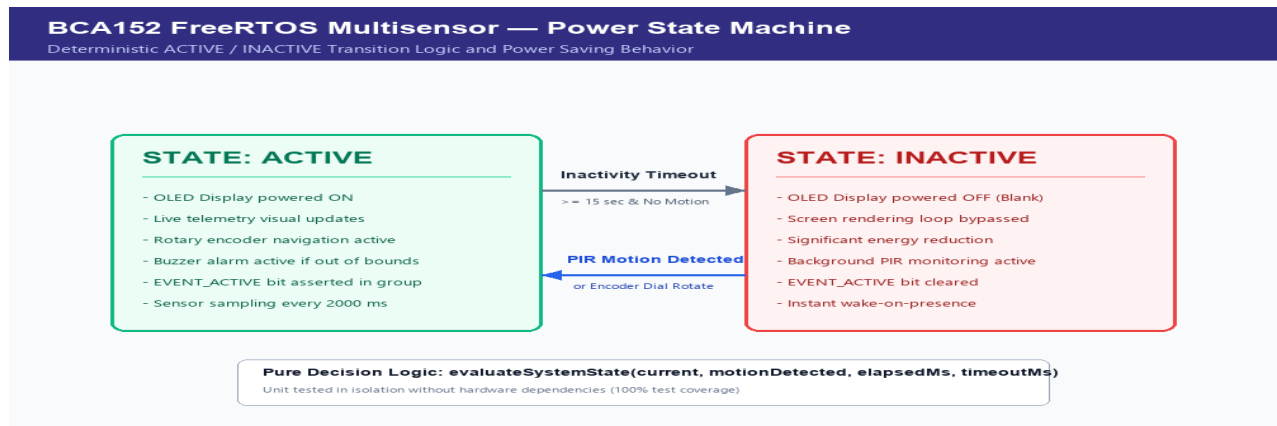


Figure 2: Finite State Machine for ACTIVE and INACTIVE power management.

3. FreeRTOS Architecture

The firmware is divided into 6 distinct FreeRTOS tasks. Instead of using shared global variables without protection, tasks communicate safely using FreeRTOS queues, event groups, and a mutex.

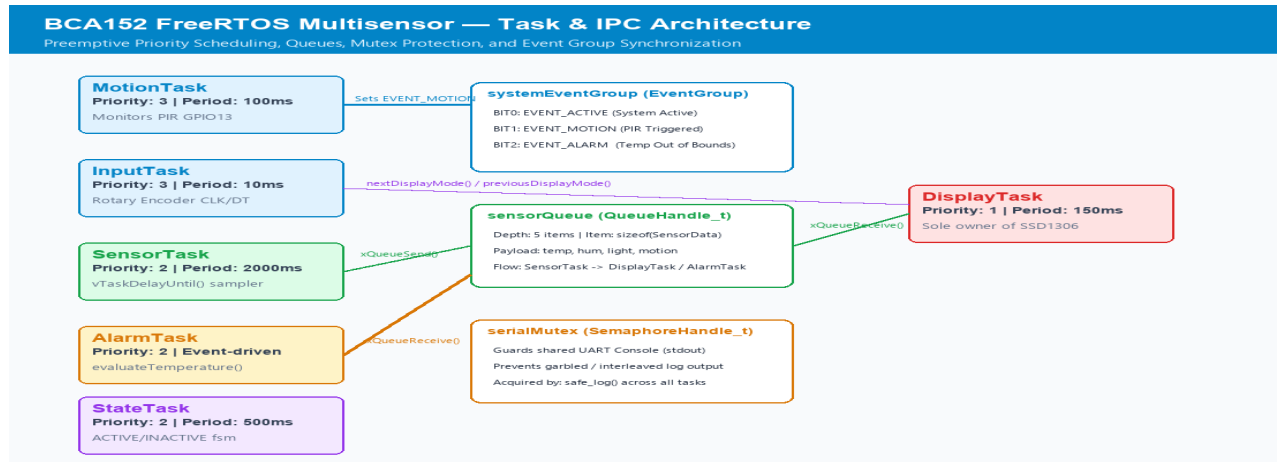


Figure 3: FreeRTOS task communication, queue dataflow, and synchronization.

FreeRTOS Task Schedule Table:

Task	Responsibility	Trigger / Period	Priority	IPC	Typical Blocked Condition
MotionTask	Checks PIR sensor for motion	100 ms periodic	3	Event group (EVENT_MOTION)	vTaskDelay(100ms)
InputTask	Decodes rotary encoder turns	10 ms periodic	3	Mode update / Direct	vTaskDelay(10ms)
SensorTask	Reads DHT22 & LDR sensors	2000 ms (2 s)	2	Queue (sensorQueue)	vTaskDelayUntil(&wake;, 2s)
AlarmTask	Evaluates temperature & buzzer	Sensor update	2	Queue (sensorQueue)	xQueueReceive(sensorQueue)
StateTask	Manages ACTIVE/INACTIVE timeout	500 ms periodic	2	Event group (EVENT_ACTIVE)	vTaskDelay(500ms)
DisplayTask	Sole owner of SSD1306 OLED	150 ms periodic	1	Queue / Shared state	vTaskDelay(150ms)

Task Priority Justification:

- **Priority 3 (InputTask, MotionTask):** Highest application priority. These tasks handle direct user interaction (rotary encoder) and physical motion events. If they had low priority, rapid knob turns would be missed or feel laggy.
- **Priority 2 (SensorTask, AlarmTask, StateTask):** Medium priority. Sensor readings happen every 2 seconds, and AlarmTask wakes up as soon as a packet arrives in the queue to check if an alarm should sound.
- **Priority 1 (DisplayTask):** Lowest priority. Updating the OLED over I2C takes several milliseconds. Since human eyes cannot notice a few milliseconds of screen delay, keeping it at Priority 1 ensures the display never blocks user inputs or alarms.

Why vTaskDelayUntil() is Used for SensorTask:

Normal `vTaskDelay(2000)` adds a 2-second sleep *after* the sensor reading finishes. If reading the sensor takes 100 ms, the total cycle becomes 2100 ms. Over time, this causes cumulative timing drift. In contrast, `vTaskDelayUntil(&lastWakeTime, 2000)` calculates the next execution time based on the previous wake time. This keeps the sampling rate steady at exactly 2.0 seconds regardless of how long sensor acquisition takes.

4. Implementation Decisions

- **Pure Decision Functions:** All core logic functions (`evaluateTemperature`, `nextDisplayMode`, `previousDisplayMode`, and `evaluateSystemState`) were written as pure functions that take inputs and return results without touching hardware pins directly. This allowed me to easily test them in automated unit tests.
- **Single OLED Owner:** If multiple tasks try to write to the SSD1306 screen at the same time over I2C, data packets collide and corrupt the screen. To prevent this, DisplayTask is the only task allowed to talk to the OLED.
- **Serial Mutex Protection:** Multiple tasks print status messages to the terminal. Without synchronization, text lines got cut off and mixed together. I wrapped terminal prints in a binary mutex (`serialMutex`) so that only one task prints at a time.
- **Pre-build Git Fix:** When compiling with ESP-IDF, CMake failed due to an empty Git reference in the Windows user directory. I wrote a Python script (`pre_build_git_fix.py`) configured in `platformio.ini` to automatically ensure the reference exists before building.

5. Verification and Testing

Automated Unit Tests (13 Tests in Unity):

Test Name	Category	Input Stimulus	Expected Result	Status
test_temperature_below_low_limit	Alarm	17.9 °C (< 18.0 °C)	LOW_TEMPERATURE	PASS
test_temperature_at_low_limit	Alarm	18.0 °C (Boundary)	NORMAL	PASS
test_temperature_normal_mid	Alarm	24.5 °C (Normal)	NORMAL	PASS
test_temperature_at_high_limit	Alarm	30.0 °C (Boundary)	NORMAL	PASS
test_temperature_above_high_limit	Alarm	30.1 °C (> 30.0 °C)	HIGH_TEMPERATURE	PASS
test_navigation_forward_step	Navigation	TEMP → Rotate CW	HUMIDITY	PASS
test_navigation_reverse_step	Navigation	HUMIDITY → Rotate CCW	TEMPERATURE	PASS
test_navigation_forward_wraparound	Navigation	MOTION → Rotate CW	TEMPERATURE (Wrap)	PASS
test_navigation_reverse_wraparound	Navigation	TEMP → Rotate CCW	MOTION (Wrap)	PASS
test_system_state_active_no_timeout	State	ACTIVE, idle 5s (< 15s)	ACTIVE	PASS
test_system_state_active_timeout_reached	State	ACTIVE, idle 15s (≥ 15s)	INACTIVE	PASS
test_system_state_inactive_no_motion	State	INACTIVE, no motion	INACTIVE	PASS
test_system_state_inactive_motion_detected	State	INACTIVE, motion = true	ACTIVE	PASS

Test Result Summary: 13 Tests Run, 0 Failures, 0 Ignored (100% Passed).

Wokwi Functional Verification (FT-01 to FT-10):

ID	Stimulus / Action	Expected Behavior	Actual Observed Behavior	Result
FT-01	Set DHT22 temp to 28.0 °C	OLED shows 28.0 °C	OLED updated to 28.0 °C on next 2s cycle	PASS
FT-02	Set DHT22 humidity to 75%	OLED shows 75.0%	OLED updated humidity value accurately	PASS
FT-03	Adjust LDR light slider	OLED light percentage updates	Light reading changed smoothly from 0 to 100%	PASS
FT-04	Rotate encoder clockwise	Next menu page displayed	Switched: Temp → Humidity → Light → Motion	PASS
FT-05	Rotate encoder counter-clockwise	Previous page with wraparound	Traversed pages in reverse order smoothly	PASS
FT-06	Set temperature to 32 °C (> 30 °C)	Buzzer turns ON, OLED shows alarm	Buzzer beeps and *ALARM* text displayed	PASS
FT-07	Return temperature to 24 °C	Buzzer turns OFF, status normal	Buzzer stopped sounding immediately	PASS
FT-08	Click PIR sensor to trigger motion	EVENT_MOTION set, system stays ACTIVE	Motion icon appeared, timer reset	PASS
FT-09	Wait 15 seconds without motion	System enters INACTIVE, OLED turns off	OLED went blank at exactly 15 seconds	PASS
FT-10	Trigger PIR while INACTIVE	System wakes up to ACTIVE, OLED on	OLED turned back on instantly with data	PASS

Deliberate FreeRTOS Fault Experiments:

- **Experiment 1 — Removing Task Delay:** I temporarily removed `vTaskDelayUntil()` from `SensorTask`. Because it ran continuously in an infinite loop without yielding, it consumed 100% of CPU time on its core. Lower-priority tasks could not run, and after 5 seconds the hardware Task Watchdog Timer triggered a reboot. This proved why every continuous task must block.
- **Experiment 2 — Changing Task Priority:** I raised `DisplayTask` from Priority 1 to Priority 4 (higher than `InputTask`). When rotating the encoder dial, page changes became laggy and missed clicks because the slow I2C display updates preempted the encoder. This proved why UI displays should stay at low priority.
- **Experiment 3 — Removing Mutex Protection:** I removed `serialMutex` from terminal printing. When multiple tasks printed at the same time, text lines overlapped and got corrupted. Restoring the mutex solved this.

6. Static Code Analysis

I ran static code analysis using PlatformIO's built-in pio check tool (which uses cppcheck). The final analysis passed with **0 defects**.

Check / Warning	File / Location	Severity	Cause / Meaning	Resolution
unusedFunction: app_main	src/main.cpp	Style	Reported because it is called by the ESP-IDF bootloader	Added extern "C" declaration
toolchain syntax error	ESP-IDF headers	Warning	Cppcheck scanned internal toolchain header files	Added check_skip_packages = yes in platformio.ini
uninitMemberVar	src/sensors.cpp	Medium	Struct variables had uninitialized fields	Used {} to zero-initialize all structs

7. Engineering Discussion

Trade-Offs and Design Choices: Using FreeRTOS on the ESP32 introduces a small amount of memory overhead for task stacks and context switching, but it provides huge advantages over a traditional Arduino superloop. By giving InputTask priority 3 and DisplayTask priority 1, the user interface remains completely responsive even while the screen is rendering or the sensor is sampling.

Problems Encountered and Solutions:

1. *CMake Git Reference Error:* At the start of the build, CMake failed because of an empty Git reference in my user folder. I resolved this by adding pre_build_git_fix.py.
2. *OLED Bus Contention:* If multiple tasks printed directly to the screen, I2C transfers conflicted. Making DisplayTask the sole owner of the display completely fixed this issue.
3. *Terminal Garbling:* Unsynchronized printf() calls produced overlapping text. Adding serialMutex ensured clean diagnostic logs.

Requirements Traceability Matrix:

Req ID	Requirement Description	Implemented In	Verified By
FR-01	Periodic temperature measurement	SensorTask in sensors.cpp	Unit Tests 1–5, Functional Test FT-01
FR-02	Periodic humidity measurement	SensorTask in sensors.cpp	Functional Test FT-02
FR-03	Ambient light monitoring (0–100%)	ldr_read_light_level() (ADC1)	Functional Test FT-03
FR-04	PIR motion detection	MotionTask in motion.cpp	Functional Test FT-08
FR-05	OLED display of sensor data	DisplayTask in display.cpp	Functional Tests FT-01 – FT-03
FR-06	Rotary encoder 4-page navigation	InputTask in input.cpp	Unit Tests 6–9, Functional Tests FT-04, FT-05
FR-07	Temperature alarm buzzer (< 18°C or > 30°C)	AlarmTask in alarm.cpp	Unit Tests 1–5, Functional Tests FT-06, FT-07
FR-08	ACTIVE and INACTIVE system states	evaluateSystemState()	Unit Tests 10–13, Functional Tests FT-08 – FT-10
FR-09	15s inactivity automatic power save	StateTask in system_state.cpp	Unit Test 11, Functional Test FT-09
FR-10	Automatic reactivation on motion	PIR motion trigger & event group	Unit Test 13, Functional Test FT-10

8. Conclusion

Through this laboratory activity, I gained a practical understanding of embedded real-time programming with FreeRTOS. I learned how to structure tasks, use queues for inter-task communication, protect shared resources with mutexes, and synchronize events using event groups. The deliberate fault experiments clearly showed what happens when tasks starve the CPU or when priorities are incorrectly assigned.

All 10 functional requirements (FR-01 to FR-10) were successfully built, tested with 13 automated unit tests, and verified in Wokwi. For future work, this system could be improved by connecting the ESP32 to Wi-Fi to publish environmental data via MQTT, and implementing deep sleep mode to further minimize power consumption when inactive.